The amputable substrate: agent runtimes designed against the bitter lesson

Hugo O'Connor · `codeberg.org/anuna` · 2026-04-28 *Position paper · CC-BY-4.0*

Abstract

Most LLM-agent frameworks today embed assumptions about what models can or cannot do — planner modules, prompt-template engines, output parsers, hand-written tool-dispatch logic. Such assumptions age badly under Sutton's bitter lesson: as model capability climbs, scaffolding becomes obsolete and the framework's value collapses into migration burden. We argue that agent runtimes should commit to *operational scaffolding only* — supervision, identity, audit, capability routing — and treat every other layer as an *amputable substrate* that the user can redefine in flight. Three principles follow: every layer redefinable at runtime without restart, the runtime artifact is the heap, and safety comes from invariants, not pipelines. We offer **anuna-imago**, a ~4100-LOC SBCL Common Lisp harness, as an existence proof. We acknowledge the wager openly: if model capability plateaus, the prescriptive frameworks were correct to bake in scaffolding.

1. Introduction

What should an LLM-agent runtime commit to? The mainstream answer over the past three years has been to give the developer a planner, a memory module, a tool selector, a prompt-template engine, and a curated dispatch loop. These abstractions answer the question "*what does the model need help doing?*" — and in answering it, they commit the framework to a specific theory of contemporary model limitation. The thesis of this paper is that such commitments are wagers against capability scaling, and they have been losing.

Concrete examples — without naming particular projects, since the structural argument is independent of any one of them — are easy to find. Frameworks that shipped explicit planner modules in 2022 found that 2023's chain-of-thought tool-use loops did the same work in less code. Frameworks that shipped regex or grammar-based output parsers in 2023 found that 2024's structured-output modes obsoleted them in months. Frameworks that built elaborate prompt-template engines for fragile instruction-following found that the templating value shrank as instruction-following improved. In every case the framework's abstractions encoded a model limitation, the limitation lifted, and the abstraction became dead weight or migration debt.

The alternative we propose is to refuse, at the framework layer, to encode assumptions about model capability. Commit only to *operational scaffolding* — supervision, identity, audit, capability routing, image distribution, runtime safety invariants. These are about how the agent process *exists and is governed*, not about what work it does. They are stable across model generations because they are not predictions about model behaviour at all.

This stance has consequences. Three principles structure them: every layer must be redefinable at runtime without restart (the user must be able to amputate framework decisions in flight), the runtime artifact must be the heap (so live redefinitions survive deployment), and safety must come from invariants rather than from prescriptive pipelines (so the safety contract does not need to extend each time the model gains a capability). The remainder of this paper develops these principles (§3), connects them to the bitter lesson (§2), offers anuna-imago as a concrete existence proof (§4), positions the stance against adjacent traditions (§5), and concludes by acknowledging the wager (§6).

2. The bitter lesson, applied to runtimes

Sutton's *bitter lesson* [Sutton 2019] holds that across game-playing, speech, vision, and translation, "general methods that leverage computation are ultimately the most effective, and by a large margin." The history of each subfield has been one of researchers building elaborate domain heuristics, only to be overtaken by simpler methods that scaled with compute. Sutton's argument concerns *research methods* — what to spend a PhD or a lab budget on. We argue the lesson transfers cleanly to *engineering frameworks*: abstractions that compensate for current model limitations are themselves hand-engineering, and they age out in the same way and for the same reason.

Three concrete examples make the transfer concrete. **Explicit planner modules** were a major offering of 2022-era agent frameworks; they encoded a theory that LLMs could not decompose tasks without help. Tool-using chain-of-thought loops have since done the same work directly, and the explicit-planner modules now exist mainly to be skipped. **Output parsers** — regex, grammar, BNF — were necessary infrastructure when models would not reliably produce valid JSON. Within roughly a year, structured-output and tool-calling modes made the parsers near-vestigial; the libraries have become legacy maintenance for code paths most users no longer enter. **Prompt-templating engines** built around the assumption that instruction-following was fragile have shrunk in value as instruction-following improved; the abstractions remain in framework code but the user no longer needs them.

A natural counterargument is that *every* framework needs *some* opinion — the choice is not between opinionated and unopinionated but between which opinions to hold. We accept the framing and refine the claim: framework opinions about *operational concerns* (process supervision, audit, distribution, identity, capability routing) are stable across model generations because they are not predictions about model behaviour at all. Framework opinions about *capability concerns* (what the model needs help doing) are unstable because they are exactly such predictions. The advice is not "have no opinions" but "hold the opinions that don't depend on contingent facts about today's models."

The migration cost of capability-tracking opinions is underdiscussed. Each time a model generation obsoletes a framework abstraction, engineering teams choose between rewriting against the new framework idiom (paying migration debt) or carrying dead code paths (paying maintenance debt). Neither outcome rewards the original choice. A framework that confines itself to operational concerns *cannot owe* this migration debt, because the operations themselves did not change.

3. The amputable substrate

We use *amputable substrate* to name a runtime in which every layer is removable, replaceable, or redefinable in flight. The framework provides infrastructure for the choices that don't depend on model capability; the user provides the choices that do. Three principles make the term concrete.

3.1 Every layer redefinable at runtime without restart

The framework treats methods as dispatch points, not call sites. Tool selection, hook handling, provider streaming, prompt assembly — each is a named operation whose implementation can be replaced while the system runs. The lineage runs through McCarthy's metacircular `eval` [McCarthy 1960] and its descendants in Lisp and Smalltalk: the language can interpret and modify its own representation. When a built-in in the framework turns out to encode an obsolete assumption, the user redefines it at the live REPL instead of filing a framework migration ticket. The framework cannot bake "the right way to do X" into a closed implementation; instead it exposes the seam where X is dispatched and offers a default. This is more demanding than configurability — configuration assumes the framework anticipated the axes of variation. Late binding does not.

3.2 The runtime artifact is the heap

Distribution is image distribution. The deployed binary is a frozen heap, including any patches the user applied at the live REPL before the snapshot was taken. There is no separate deployment pipeline whose assumptions might themselves age out — there is no pipeline, only the heap. A consequence is that the redefined-in-flight semantics survives deployment: the binary behaves exactly as the running image did at the moment of save. SBCL's `save-lisp-and-die` produces a single-file executable in this style [Steele & Gabriel 1996], descended from the Lisp-Machine and Smalltalk image traditions [Goldberg & Robson 1983]. The implication for framework design is that any tool the framework offers must be safe to freeze mid-flight: open files, credentials, network handles must either be cleaned before save or tolerate restoration on resume.

3.3 Safety from invariants, not pipelines

Safety is not a sequence of checks the framework runs in a fixed order; it is a set of statements about the system that must always hold. The framework gates on whether invariants are violated, but does not own the invariants themselves. The user authors them — in this implementation, in defeasible logic — and the framework consults them at the relevant points. The practical consequence is that when the model gains a new capability, the invariants don't need to change. A pipeline-based safety story, by contrast, gets longer with every new capability surface: a new tool means a new validator, a new orchestration step, a new place to forget a check. An invariant-based story stays the same size: the new capability either does or does not violate the existing rules. anuna-imago's SPEC-012 self-modification port queries a Spindle defeasible-logic theory at every `harness-eval` call; the theory is the entire safety contract for that surface, and the framework's role is reduced to "ask the reasoner, veto on positive verdict."

4. anuna-imago as existence proof

We have built a runtime that follows the three principles, called **anuna-imago**. The implementation is approximately 4100 lines of Common Lisp on SBCL with a further 3340 lines of tests; the harness is small enough to read in an afternoon, and small enough that no part of it is load-bearing in the sense that the user cannot replace it. We offer the artifact as a demonstration that the principles cohere into a working system, not as a product pitch — the choice of substrate (SBCL Common Lisp) carries real costs that we acknowledge explicitly below.

4.1 A worked redefinition

The headline behaviour of the runtime is that any method can be redefined at the live REPL and the redefinition will survive an image save. The following ~10 lines of transcript construct an agent against a stub provider, ask it, redefine the provider's `stream!` method to produce different output, ask again (new behaviour, same process, no restart), then save the heap as an executable binary. Running the binary from the shell produces output consistent with the redefined method:

```
(defparameter *agent* (build-echo-agent))
(spawn-agent! *sup* *agent*)
(getf (ask-agent *agent* "hi") :text)        ; => "echo: hi"

(defmethod provider-stream! ((p stub-provider) agent message)
  (declare (ignore agent))
  (let* ((c (if (listp message) (getf message :content) message))
         (cell (cons nil (list (list :text (format nil "shouted: ~A!"
                                                   (string-upcase c)))))))
    (lambda () (pop (cdr cell)))))

(getf (ask-agent *agent* "hi") :text)        ; => "shouted: HI!"

(save-image! "shouty-agent" :toplevel 'agent-main)
```

```
$ ./shouty-agent --echo "hello"
shouted: HELLO!
```

No framework migration ticket; no rebuild step. The patch survives because the binary *is* the heap.

4.2 Safety from invariants in practice

The SPEC-012 self-modification port instantiates Principle 3. A `harness-eval` tool lets the agent submit Common Lisp source forms for evaluation; before evaluation, three layers gate the call: a pre-filter denylist for obvious structural violations, a defeasible-logic reasoner querying a Spindle theory of floor invariants, and a handler that runs the form under timeout with a rollback register for any methods it redefines. A published log of six goal-driven runs with GLM 5.1 records the agent self-redefining 18 symbols across 6 turns to add persistent memory to itself, with 3 vetoes from the floor invariants intercepting attempts to redefine the safety surface. The invariants are short, queryable, and authored once.

4.3 Honest costs

Common Lisp on SBCL is a small ecosystem with a sharp learning curve and genuine hiring difficulty. Teams committed to Python or TypeScript ergonomics will not adopt anuna-imago verbatim. Our argument is structural: *some* substrate that supports these principles must exist, and substrates that do not — including the typical Python-based agent stack — pay back the limitation as migration cost over time. anuna-imago demonstrates that the position is realisable; it does not claim to be the realisation everyone should choose.

5. Related stances

The position has antecedents and convergent neighbours.

The Lisp / Smalltalk image lineage [McCarthy 1960; Goldberg & Robson 1983]. The substrate spirit we adopt — `eval` / `apply` self-reference, late binding, image-as-artifact — is the inheritance of fifty years of metacircular language design. The lineage applied these properties to *integrated development*: Lisp Machines and Smalltalk-80 were workstations

where the developer and the program lived in the same heap. Our claim is that the spirit transfers to LLM-agent runtimes, and that the bitter lesson makes the transfer newly important — not because agent runtimes are a new application domain, but because they are the first application domain where the *user of the runtime* (the LLM) has capability that climbs faster than the framework's release cycle.

Anthropic's "Building Effective Agents" [Anthropic 2024]. Recent engineering writing argues empirically that simpler agent loops — plain tool-use, no orchestration framework — often outperform complex multi-agent architectures. We arrive at a convergent stance via a different argument: the bitter lesson predicts that the orchestration abstractions will pay obsolescence cost over time. Both directions point at the same practical conclusion. The convergence between an empirical engineering finding and a structural theoretical prediction is itself evidence that the position is robust.

The Erlang/OTP supervision tradition [Armstrong 2003]. anuna-imago's `make-supervisor` is a direct CLOS port of Armstrong's one-for-one supervisor, and its mailbox abstraction follows the same letterbox-with-pid shape as Erlang processes. The contribution we claim is *not* the supervisor — Armstrong's design is fully borrowed and unmodified. The contribution is to limit framework scope to such established prior art and push every other concern to user code. Operational scaffolding is not a new research problem; what is new is committing to it as the *only* framework concern.

6. Conclusion

Agent runtimes should commit to operational scaffolding only — supervision, identity, audit, capability routing — and treat every other layer as an amputable substrate the user can redefine in flight. Three principles implement the position: live redefinition, image-as-artifact, invariants not pipelines. anuna-imago, ~4100 LOC of Common Lisp, demonstrates that the position is realisable. The wager is plain: if model capability plateaus in the next several years, prescriptive frameworks were correct to bake in scaffolding and we paid for under-engineering. If capability continues climbing, prescriptive frameworks pay migration debt forever and the substrate position wins by attrition. The artifact is at `codeberg.org/anuna/imago`. Take it apart.

References

Anthropic (2024). *Building Effective Agents.* Engineering blog post, `anthropic.com/engineering/building-effective-agents`.

Armstrong, J. (2003). *Making reliable distributed systems in the presence of software errors.* PhD thesis, KTH.

Goldberg, A. & Robson, D. (1983). *Smalltalk-80: The Language and its Implementation.* Addison-Wesley.

McCarthy, J. (1960). Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I. *Communications of the ACM* 3(4):184–195.

Steele, G. & Gabriel, R. (1996). The evolution of Lisp. *History of Programming Languages II.* ACM Press.

Sutton, R. (2019). *The Bitter Lesson.* `incompleteideas.net/IncIdeas/BitterLesson.html`.